

Efficient Path Planning with Neural Networks

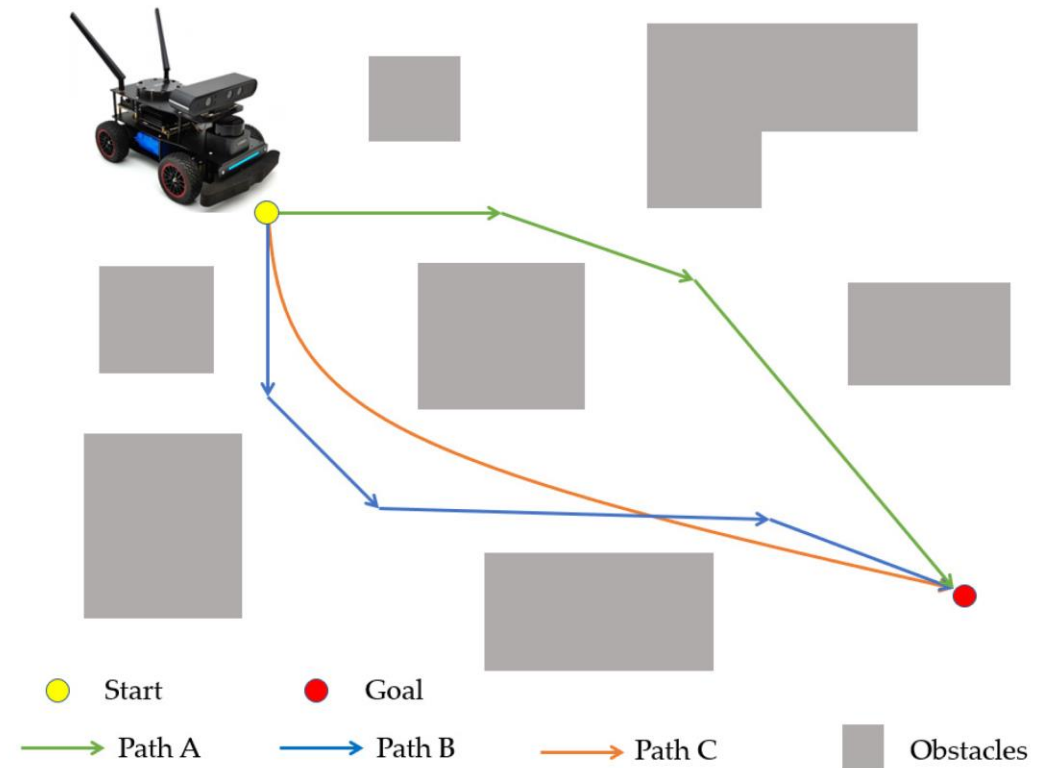
Quinten Pienaar

Agenda

- Background
- Problem Statement
- Approach
- Implementation
- Results
- Conclusion

Background – Path Planning

Path planning involves finding a collision-free course, or path, that moves an agent from a starting configuration to a goal configuration within a specific environment.



Background – Path Planning

A path planning algorithm is complete if it exhibits two key properties:

1. If a path exists, the optimal path will be found
2. If a path does not exist, this fact will be discovered

Astar is the gold standard for a complete and efficient algorithm

Background - Astar

- Visits nodes based on distance from current node and estimated/heuristic distance from goal

$$f = g + h$$

g – distance from current node

h – heuristic cost to goal

- Common heuristics: Euclidean or Manhattan distance

Background - Astar

- Astar's efficiency is governed by the quality of its heuristic
- In order to guarantee completeness, a heuristic must underestimate the distance to goal - admissibility
- In some environments traditional heuristics perform poorly



Problem Statement

- Can the efficiency Astar be improved by training a neural network to estimate heuristic cost where traditional heuristics perform poorly?
- Can we maintain admissibility, and therefore completeness, while using a learned heuristic?

Approach

- Train a convolutional neural network to estimate the distance from a point to a goal given a map using different loss functions
 - MSE, Piecewise MAE
- Randomly generate maps where Euclidean distance will perform poorly
- Quantify improvement in search cost, overestimation rate, optimality

Implementation – Neural Network

- Input:
 - 3 channels – map, query position, goal position
- Convolution:
 - Kernel 3x3, stride 1, padding 1, batch normalization, ReLu activation,
 - Layer 1: 8 filters, Max Pooling (2x2)
 - Layer 2: 16 filters
- Fully Connected Layers
- Relu activation
- Varying loss functions
- Adam optimizer
 - Layer 1: 1600 -> 256
 - 0.1 dropout rate
 - Layer 2: 256 -> 64
 - Layer 3: 64 -> 1

Implementation – Loss Functions

Two loss functions were used for training

- Mean squared error

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

- Piecewise mean absolute error [3]

$$w_i = \begin{cases} \alpha_1, & \text{if } \hat{y}_i < h_{\min,i} \\ 1, & \text{if } h_{\min,i} \leq \hat{y}_i \leq y_i \\ \alpha_2, & \text{if } \hat{y}_i > y_i \end{cases}$$

$$\mathcal{L}_{\text{piecewise}} = \frac{1}{N} \sum_{i=1}^N w_i |\hat{y}_i - y_i|$$

Implementation - Heuristic

- Given a heuristic cost estimate from a neural network, we can use it directly, or compare it with Euclidean heuristic

$$h(s, g) = \min \left(\sqrt{(x_s - x_g)^2 + (y_s - y_g)^2}, \hat{h}_\theta(s, g) \right)$$

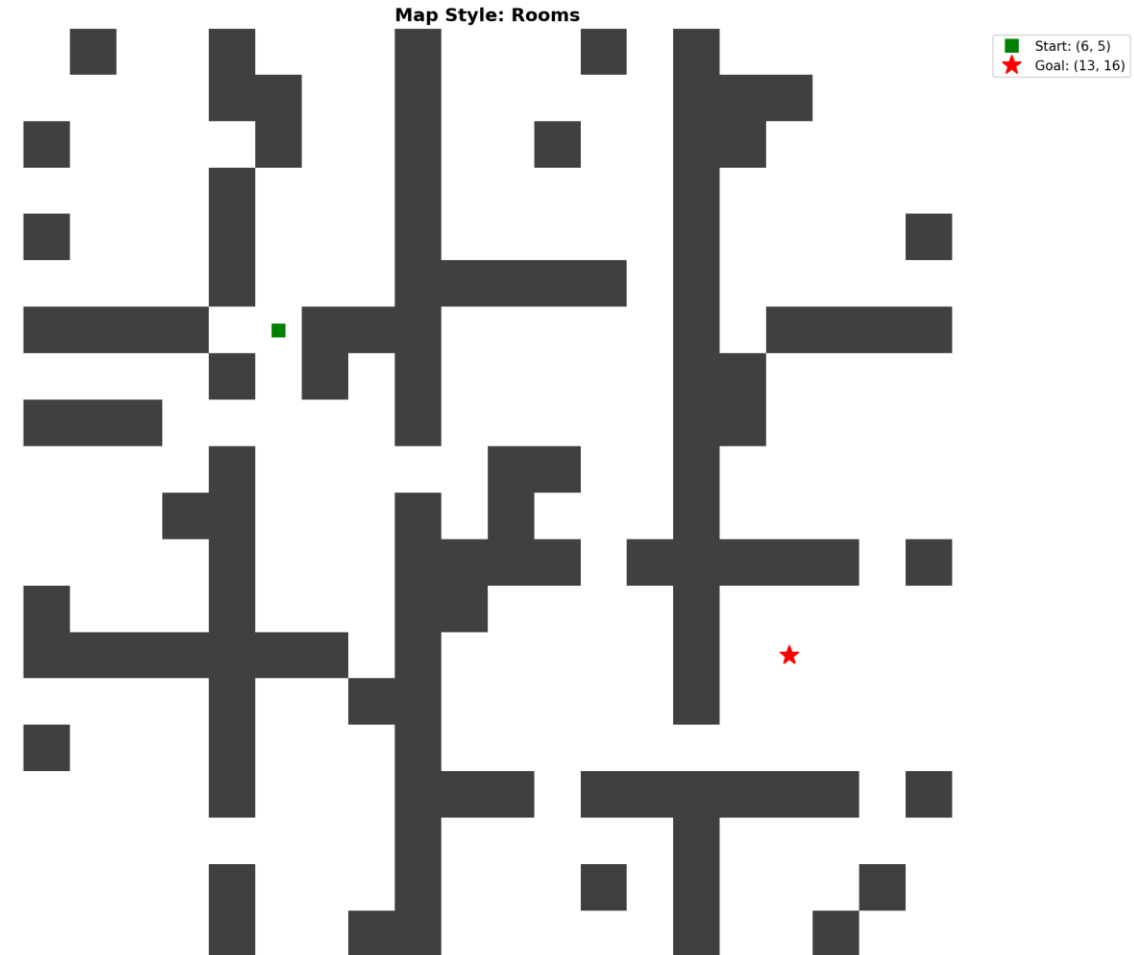
- Using heuristic directly emphasis search efficiency
- Comparing to Euclidean heuristic guarantees admissibility

Implementation – Data Generation

- 500 randomly generated 20x20 maps were created
- 80% were used for training – 20% for validation
- An additional 100 maps were generated for testing
- Maps were created according to three different styles:
 - Rooms, walls, random
 - 30 percent occupancy

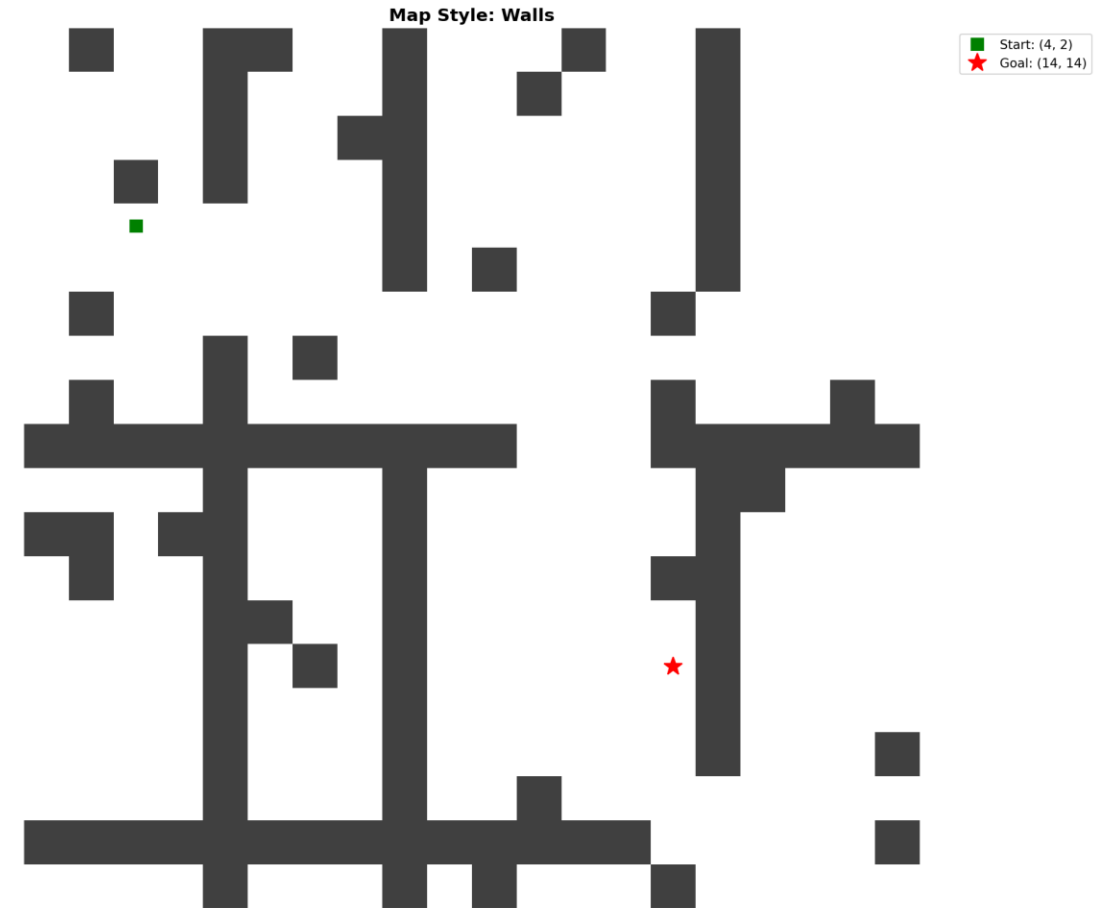
Implementation - Room

- Partition canvas into segments
- Punch openings through walls
- Add noise



Implementation - Walls

- Randomly grow walls from canvas edges towards center
- Add noise



Implementation - Random

- Pure noise



Implementation - Experiments

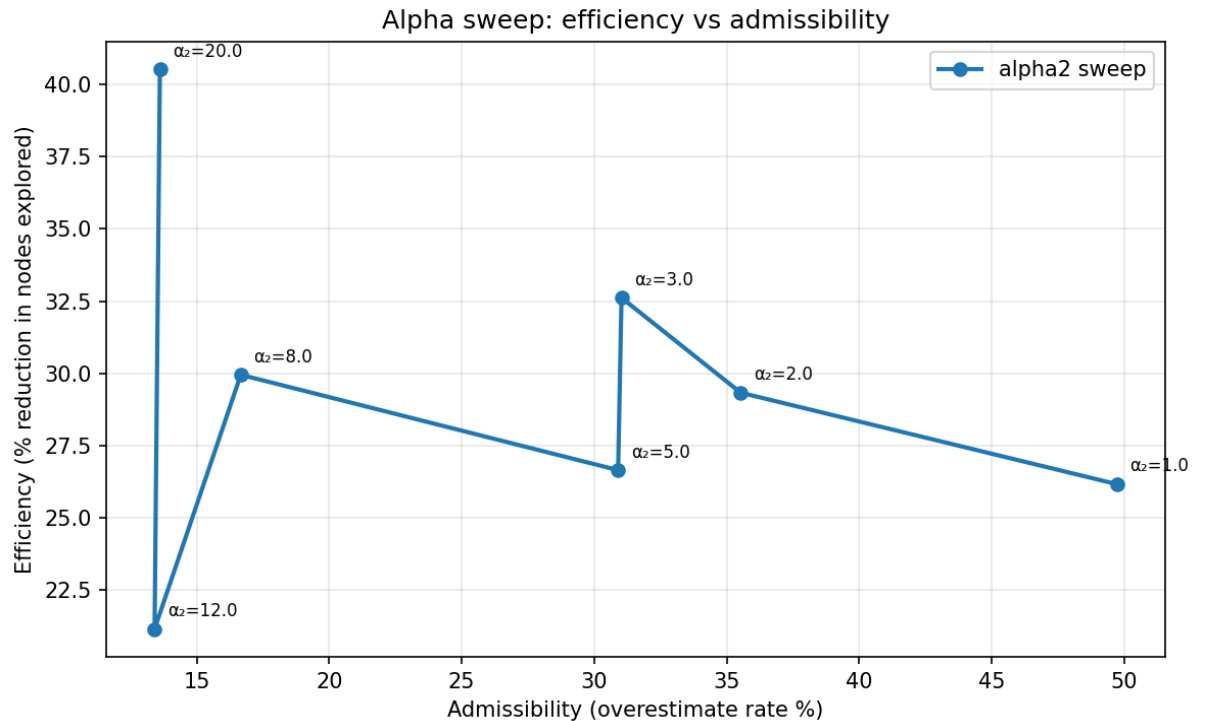
Four approaches were compared to a pure Euclidean heuristic

1. Directly use learned heuristic with MSE loss function in training
2. Directly use learned heuristic with piecewise MAE loss function in training
3. Use minimum between Euclidean and MSE learned heuristic
4. Use minimum between Euclidean and piecewise MAE learned heuristic

Each approach was tested on 100 maps while recording the reduction in search cost, overestimation rate, and optimality

Results – Optimal MAE

- Before experimentation α_2 needed to be determined for piecewise MAE loss
- CNNs with a variety of α_2 values underwent training & validation
- $\alpha_2 = 20$ was taken as the optimal value

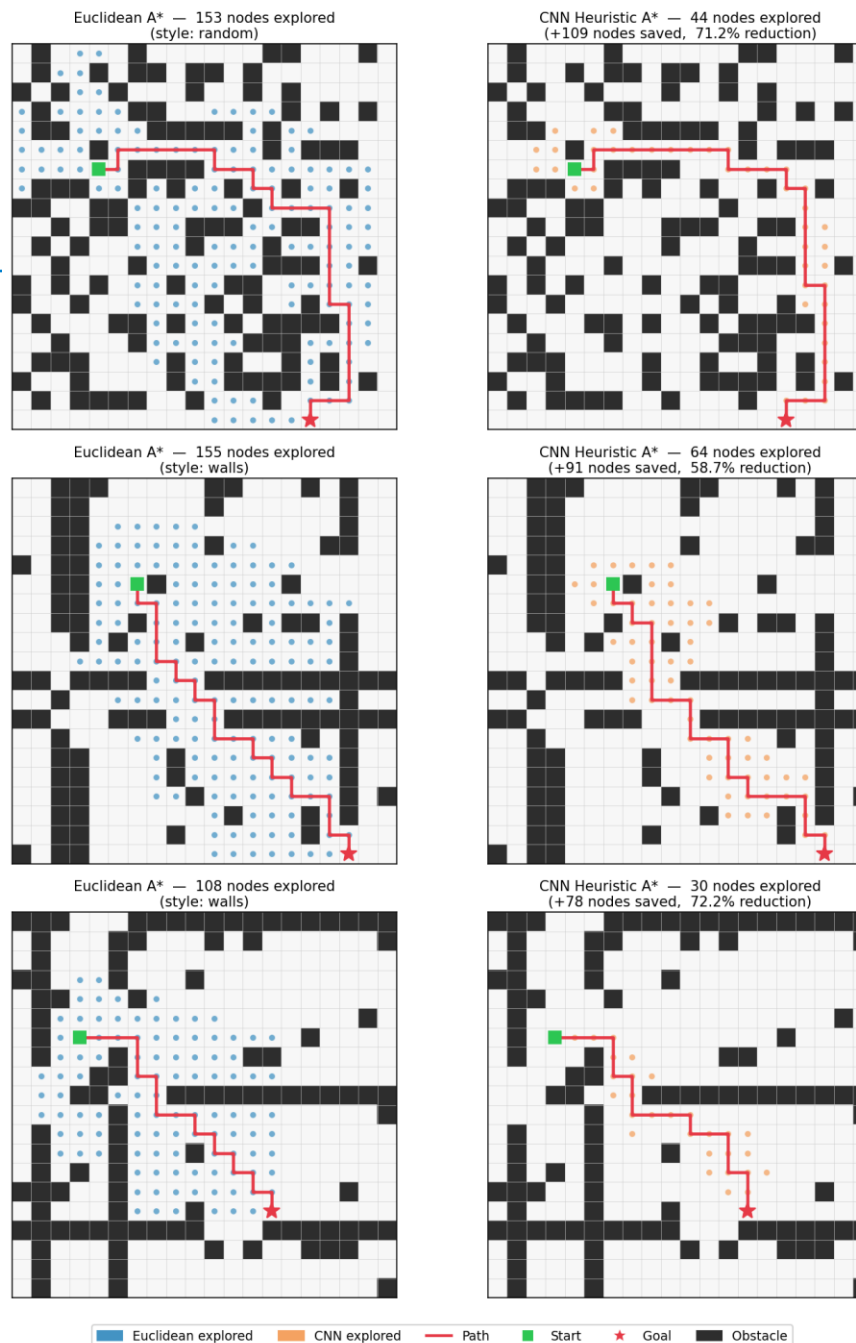


Results

Configuration	Percent Search Cost Reduction	Overestimate Rate	Optimality Rate
Euclidean heuristic	NA	0	100
Learned – MSE loss function	30.7	50.2	90
Learned – Piecewise MAE loss function	34.2	25.9	94
Minimum heuristic – Euclidean MSE	2.2	0.0	100
Minimum heuristic – Euclidean Piecewise MAE	5.5	0.0	100

Results - Visual

Vanilla A* vs CNN-Heuristic A*
(blue = Euclidean explored, orange = CNN explored)



Conclusion

- Neural networks can be used to effectively decrease search costs in path planning algorithms
- Using asymmetric loss functions can increase the effectiveness of the neural network
- A pure learned heuristic is likely to break admissibility
- To maintain admissibility, higher search costs must be accepted
- Future work can be done with more sophisticated NN, different algorithms, or real-world experiments

RPI

Sources

- [1] S. Pan, *MANE/ECSE 6961 Scientific Machine Learning Lecture Slides*, Rensselaer Polytechnic Institute, 2026.
- [2] E. Yel, *MANE 6967 Robotics II Lecture Slides*, Rensselaer Polytechnic Institute, 2026.
- [3] T. Takahashi, H. Sun, D. Tian, and Y. Wang, “Learning heuristic functions for mobile robot path planning using deep neural networks,” in *Proc. Int. Conf. Automated Planning and Scheduling (ICAPS)*, Jul. 2019, pp. 764–772.
- [4] MathWorks, “Convolutional Neural Network,” [Online]. Available: <https://www.mathworks.com/discovery/convolutional-neural-network.html>. [Accessed: Apr. 28, 2026].
- [5] A. Amini, “MIT 6.S191 (2025): Convolutional Neural Networks,” YouTube, [Online]. Available: <https://www.youtube.com/watch?v=oGpzWAlP5p0>. [Accessed: Apr. 28, 2026].